

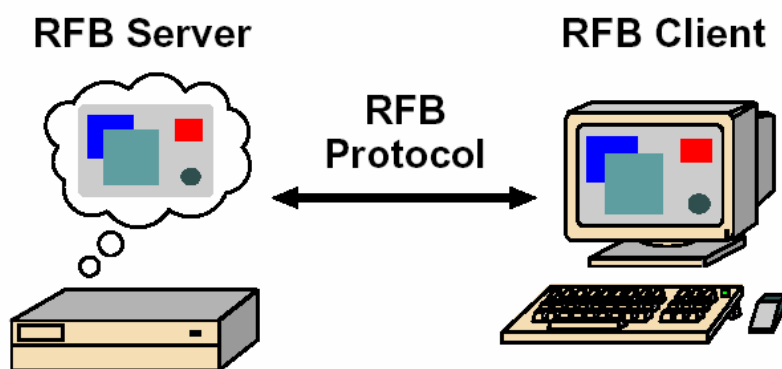
บทที่ 5

RFB Protocol

5.1 อาร์เอฟบีโพรโทคอล (RFB Protocol)

อาร์เอฟบี (RFB : Remote Frame Buffer) เป็นโพรโทคอลสำคัญที่เราจะนำมาใช้เป็นโพรโทคอลที่จะถูกนำมาใช้ในการเข้าควบคุมเครื่องคอมพิวเตอร์จากระยะไกลผ่านยูสเซอร์อินเตอร์เฟซเนื่องจากว่ามันจะทำงานในระดับของเฟรมบัพเฟอร์ มันจึงสามารถใช้ได้กับระบบและแอปพลิเคชันทั้งหมดของวินโดวส์ รวมไปถึงระบบ เอ็กซ์11 (X11), วินโดวส์ 3.1/95/เอ็นที และแมคอินทอช

ในโพรโทคอลนี้จะเรียกส่วนต่างๆดังนี้ รีโมทเอ็นด์พอยน์คือจุดที่ติดต่อกับผู้ใช้(เช่น จอ คีย์บอร์ด เมาส์) จะเรียกว่า “อาร์เอฟบีไคลเอนต์” และจุดเอ็นท์พอยน์ที่เกิดการเปลี่ยนแปลงของเฟรมบัพเฟอร์ จะเรียกว่า “อาร์เอฟบีเซิร์ฟเวอร์”



รูปที่ 5-1 การเชื่อมต่อของโพรโทคอล

อาร์เอฟบี โดยทั่วไปจะถูกเรียกว่า “ ดินไคลเอนต์ (thin client) โพรโทคอล “ ส่วนที่เป็นหลักสำคัญในการออกแบบอาร์เอฟบีโพรโทคอล คือการทำให้ไคลเอนต์มีข้อกำหนดต่างๆน้อยที่สุด นั่นก็คือไคลเอนต์สามารถที่จะทำงานบนฮาร์ดแวร์ได้หลากหลายไม่จำกัดอยู่แค่แบบหนึ่งแบบใดเท่านั้นและงานหนักของการนำโพรโทคอลไปใช้งานกับไคลเอนต์ก็คือการทำให้มันทำงานได้ง่ายที่สุดเท่าที่จะเป็นไปได้

โพรโทคอลนี้จะทำให้ไคลเอนต์ทำงานแบบสเตทเลส (stateless) คือ ถ้าไคลเอนต์หลุดการเชื่อมต่อจากเซิร์ฟเวอร์และต่อมาได้ทำการเชื่อมต่อเข้าไปใหม่ยังเซิร์ฟเวอร์ตัวเดิม สถานะของส่วนที่ติดต่อกับผู้ใช้จะต้องคงสถานะเดิมไว้อย่างที่นั่นตัวโพรโทคอลยังสามารถยอมให้ไคลเอนต์หลายๆตัวทำการเชื่อมต่อไปยังอาร์เอฟบีเซิร์ฟเวอร์ได้ทั้งที่ไคลเอนต์ผู้ใช้สามารถที่จะเห็นส่วนติดต่อกับผู้ใช้เหมือนกับที่เกิดขึ้นในฝั่งของเซิร์ฟเวอร์ผลจากข้อกำหนดนี้จะทำให้การติดต่อกับโปรแกรมใช้งานของผู้ใช้โปรแกรมจากที่อื่น (remote) กลายเป็นสิ่งที่ง่ายคายทำให้สามารถใช้งานเครื่องที่ฝั่งโปรแกรมส่วนเซิร์ฟเวอร์ที่ไหนก็ตามที่มีการเชื่อมต่อกับเน็ตเวิร์กและใช้งานยังสามารถกระทำการใดๆกับโปรแกรมใช้งานที่อยู่ในเครื่องส่วนตัว

ที่อยู่ห่างออกไปได้นอกจากนั้นสถานะของการเข้ากระทำนี้จะถูกเก็บรักษาไว้ระหว่างที่มีการกระทำจากที่อื่น

5.2 โพรโตคอลของการแสดงภาพ

ในส่วนแสดงภาพของโพรโตคอลนี้ ใช้พื้นฐานการแสดงรูปภาพอย่างหยาบๆ คือ ใช้การวางจุดของพิกเซลที่ตำแหน่ง เอ็กซ์,วาย

ในขั้นตอนแรกนี้อาจจะมองวิธีที่จะวาดส่วนประกอบหลายๆส่วน ของส่วนติดต่อผู้ใช้ อย่างไรก็ดีตามหากรูปแบบการเข้ารหัสของเตทพิกเซลมีความแตกต่างกันก็จะทำให้เกิดความยุ่งยากจากค่าพารามิเตอร์ต่างๆที่เกิดขึ้น อย่างเช่น แบนด์วิดธ์ของเน็ตเวิร์ก ความเร็วในการวาดของไคลเอนต์และความเร็วในการประมวลผลของเซิร์ฟเวอร์

ลำดับการในการจัดวางรูปสี่เหลี่ยมเหล่านีที่ประกอบกันขึ้นทำให้เกิดการเปลี่ยนแปลงของเฟรมบัฟเฟอร์ (หรือเรียกว่า อัปเดต) การอัปเดตเป็นการเปลี่ยนแปลงจากสถานะการใช้งานเฟรมบัฟเฟอร์หนึ่งไปยังสถานะอื่น ในอีกนัยหนึ่งก็คือมันจะคล้ายกับเฟรมของวิดีโอที่ส่งออกมาเป็นลำดับ ในกรณีทั่วๆไปแล้วเฟรมเหล่านี้จะต้องติดกันแต่ก็ไม่จำเป็นในกรณีของเรื่องนี้

โพรโตคอลในการอัปเดต จะถูกกระตุ้นโดยไคลเอนต์ นั่นคือการอัปเดตจะถูกส่งจากเซิร์ฟเวอร์ไปยังไคลเอนต์เท่านั้นในการตอบสนองความต้องการของไคลเอนต์ ในระบบที่ไคลเอนต์และเน็ตเวิร์กมีความช้ามากการอัปเดตก็จะช้าลงด้วย ในแอปพลิเคชันปกติทั่วๆไปการเปลี่ยนแปลงที่พื้นที่เดียวกันของเฟรมบัฟเฟอร์จะทำให้มีการอัปเดตเร็วกว่ากรณีอื่นๆ ในระบบที่คอมพิวเตอร์หรือ/และระบบเครือข่ายมีความช้าสถานะทรานส์เซียนอาจถูกยกเลิกได้ซึ่งจะทำให้เกิดทราฟฟิกและการวาดภาพน้อยลงในไคลเอนต์

5.3 อินพุทโพรโตคอล

ในส่วนอินพุทของโพรโตคอลจะขึ้นอยู่กับมาตรฐานของอุปกรณ์คีย์บอร์ดและอุปกรณ์จำพวกชี้ตำแหน่ง อีเวนต์ของอินพุทจะถูกส่งไปยังเซิร์ฟเวอร์โดยผ่านทางไคลเอนต์ เมื่อไหร่ก็ตามที่ผู้ใช้กดปุ่มหรือกดเมาท์ หรือมีการเคลื่อนที่ของเมาท์เกิดขึ้นก็จะถูกส่งออกไปด้วย อีเวนต์ของอินพุทเหล่านี้สามารถที่จะถูกสังเคราะห์จากอุปกรณ์อื่นที่ไม่เป็นอุปกรณ์มาตรฐาน ตัวอย่างเช่น อุปกรณ์ที่ใช่เขียนแทนปากกาจะมีตัวจดจำที่ทำหน้าที่เปลี่ยนให้เป็นอีเวนต์ที่คล้ายกับอีเวนต์ที่มาจากคีย์บอร์ด

5.4 การนำเสนอของพิกเซลเตท

ในการเริ่มต้นของการโต้ตอบกันระหว่าง อาร์เอฟบีไคลเอนต์และเซิร์ฟเวอร์จะต้องมีการตกลงกันในเรื่องของรูปแบบ (format) และการเข้ารหัส (encode) ของข้อมูลที่จะทำการส่งออกไปก่อน ข้อตกลงเหล่านี้จะถูกออกแบบมาเพื่อให้ไคลเอนต์ทำงานง่ายที่สุดเท่าที่เป็นไปได้ อย่างเช่นเส้นล่างสุดของเฟรมบนเครื่องเซิร์ฟเวอร์จะต้องสามารถส่งเตทพิกเซลให้อยู่ในรูปแบบที่ไคลเอนต์ต้องการได้ อย่างไรก็ดีตามไคลเอนต์จะต้องสามารถจัดการกับรูปแบบและการเข้ารหัสที่หลากหลายได้เท่าๆกันโดยมันจะทำการเลือกวิธีที่ง่ายสำหรับการจัดการของเซิร์ฟเวอร์

รูปแบบของจุดภาพ (Pixel format) มาจากการนำเสนอจุดของสีเดี่ยวๆ ด้วยค่าพิกเซลโดยทั่วไป แล้วรูปแบบของพิกเซลจะมีขนาดอยู่ที่ 24 บิต หรือ 16 บิต ค่าภายในบิตฟิลด์จะถูกแปลงออกโดยตรงเป็นสีแดง สีเขียวและสีน้ำเงินเข้ม และที่ความละเอียดขนาด 8 บิต เรียกว่า “ชุดสี (color map)” “จะเป็นสีที่ไม่มี การกำหนดในชุดสีมาตรฐานสามารถทำการแปลงจากค่าของพิกเซลเป็นค่าของสีต่างๆได้เลย

การเข้ารหัส(Encoding)หมายถึงการทำให้กรอบของข้อมูลพิกเซลที่ถูกส่งออกไปบนสายสัญญาณ สามารถทำการส่งได้อย่างรวดเร็ว ทุกๆชุดพิกเซลที่ถูกส่งออกไปจะต้องถูกปะหั่วด้วยด้วยแฮคเตอร์ที่บอก ตำแหน่งเอ็กซ์, ตำแหน่งวายบนหน้าจอ ความกว้างยาวของกรอบสี่เหลี่ยมและชนิดของการเข้ารหัสจะเป็น ตัวกำหนดรูปแบบการเข้ารหัสของข้อมูลของพิกเซลซึ่งโดยตัวของข้อมูลเองแล้ว มันจะอนุญาตให้มีการ กำหนดการเข้ารหัสได้

โปรโตคอลนี้สามารถทำขยายออกได้โดยการเพิ่มชนิดของการเข้ารหัสใหม่ ชนิดของการเข้ารหัส ที่ใช้อยู่ในปัจจุบัน คือ การเข้ารหัสแบบหยาบ (raw encoding) การเข้ารหัสแบบการคัดลอกกรอบ (copy rectangle encoding) การเข้ารหัสแบบอาร์อาร์อี (RRE : rise-and-run-length) การเข้ารหัสแบบซีโออาร์อาร์ อี (CoRRE : Compact RRE) และการเข้ารหัสแบบเฮกซ์ไทล์ (hextile encoding) แต่ในเรื่องนี้เราจะใช้ การเข้ารหัสแบบ เฮกซ์ไทล์และแบบการคัดลอกกรอบเท่านั้นเนื่องจากว่าการเข้ารหัสแบบนี้ได้ถูกทำให้ เป็นวิธีที่ดีที่สุดในการบีบอัดข้อมูลสำหรับเครื่องคอมพิวเตอร์ที่ใช้งานทั่วไป ตัวอย่างอื่นๆของการเข้ารหัส ที่เป็นไปได้ เช่น การเข้ารหัสแบบ JPEG ที่ใช้ส่งข้อมูลภาพและ MPEG สำหรับการส่งภาพเคลื่อนไหว

5.4.1 การเข้ารหัสแบบหยาบ (Raw encoding)

เป็นแบบที่ง่ายที่สุดของการเข้ารหัสโดยข้อมูลจะมีลักษณะเป็นข้อมูลพิกเซลแบบหยาบๆ เท่านั้น ในตัวอย่างนี้ให้ข้อมูลประกอบด้วย ค่า n พิกเซล โดยที่ n เป็นความกว้างคูณกับความสูงของกรอบ ค่าที่ได้ จะถูกนำมาใช้แทนแต่ละพิกเซล จากซ้ายไปขวาตามเส้นแนวนอน อาร์เอฟบีไคลเอนต์ทุกตัวจะต้องสามารถที่จะ จัดการกับข้อมูลพิกเซลในการเข้ารหัสแบบนี้ได้ และอาร์เอฟบีเซิร์ฟเวอร์ควรจะสร้างการเข้ารหัสแบบ นี้เท่านั้นถ้าไคลเอนต์ไม่กำหนดการเข้ารหัสแบบอื่นๆเป็นพิเศษ

5.4.2 การเข้ารหัสแบบคัดลอกกรอบ (Copy Rectangle encoding)

การเข้ารหัสแบบคัดลอกกรอบนี้เป็นวิธีที่ง่ายและมีประสิทธิภาพมาก จะถูกใช้เมื่อไคลเอนต์มี ข้อมูลพิกเซลเดียวกันภายในบัพเฟอร์ซ้ำกัน ในการเข้ารหัสแบบนี้จะประกอบด้วยพิกัดของเอ็กซ์ วาย โดย ตำแหน่งนี้เป็นตำแหน่งในเฟรมบัพเฟอร์ เมื่อถึงเวลา ไคลเอนต์ก็สามารถทำการคัดลอกกรอบของข้อมูล พิกเซลนี้ไปได้เลย จึงสามารถที่จะนำไปใช้ในหลายๆกรณีได้ ที่เห็นได้ชัดที่สุดคือ เมื่อผู้ใช้งานทำการ เคลื่อนย้ายหน้าต่างผ่านไปตามและเมื่อวินโดวส์มีการเลื่อนขึ้นหรือลงของหน้าต่าง จุดที่เห็นชัดน้อยที่สุด คือการใช้สำหรับการวาดข้อความหรือการทำการรูปแบบอื่นๆการที่จะให้ข้อมูลมีความกระชับสมบูรณ์ใน ตัวเซิร์ฟเวอร์ที่ฉลาดอาจสามารถส่งข้อมูลออกมาในรูปแบบใดรูปแบบหนึ่งออกมาตรงๆได้เลยและการที่ รู้ตำแหน่งของรูปแบบของข้อมูลก่อนหน้าในเฟรมบัพเฟอร์ จะทำให้การส่งข้อมูลตามลำดับของรูปแบบที่ คล้ายกันจะเกิดขึ้นโดยใช้การเข้ารหัสแบบคัดลอกกรอบได้ง่ายขึ้น

5.4.3 การเข้ารหัสแบบอาร์อาร์อี (RRE : rise-and-run-length encoding)

การเข้ารหัสแบบนี้จะแบ่งการทำงานออกเป็นสองส่วนสำคัญคือการรัน-ลองและการเข้ารหัสใน ระดับสองมิติ กรอบข้อมูลที่มีการเข้ารหัสแบบอาร์อาร์อี จะไม่เพียงแต่ถูกย่อในระดับเดียวกันหรือดีกว่า ด้วยการเข้ารหัสแบบ รัน-ลอง เท่านั้นหากแต่กรอบที่ถูกส่งออกไปจากโคลเอนต์นั้นจะถูกเข้ารหัสให้อยู่ใน รูปแบบที่สามารถที่จะนำไปแสดงผลได้ทันทีและเพิ่มประสิทธิภาพโดยการเรียกใช้กราฟิกเอ็นท์จิน (graphic engines) แบบที่ง่ายที่สุด อาร์อาร์อีมีจุดมุ่งหมายอยู่ที่การทำให้การบีบอัดขนาดเป็นไปตามความต้องการของเซิร์ฟเวอร์ แต่อาร์เอฟบีโคลเอนต์ไม่มีประสิทธิภาพเพียงพอในการบีบอัดข้อมูลให้มีความเร็ว เพียงพอที่จะรักษาประสิทธิภาพการใช้งาน ได้อย่างทันท่วงที

ความคิดพื้นฐานของ อาร์อาร์อี คือ การแบ่งส่วนของกรอบของข้อมูลพิกเซลให้อยู่ในรูปของ สี่เหลี่ยมย่อยๆ แต่ละส่วนย่อยๆ นั้นจะประกอบด้วยพิกเซลของพิกเซลค่าเดียวและการรวมของแต่ละชุด ของกรอบข้อมูลในตอนแรก การที่เราทำการแบ่งจากกรอบใหญ่ออกเป็นกรอบย่อยๆ นั้นเพื่อให้ง่ายต่อการ คำนวณ

ผลที่ได้จากการเข้ารหัสจะประกอบด้วย ค่าของพิกเซลที่เป็นแบ็คกราวด์ v_b (โดยปกติจะเป็น พิกเซลที่มีค่ามากที่สุดในการอบนั้นๆ) และเลขจำนวนนับ N ซึ่งจะใช้เป็นตัวแทนจำนวนของกรอบย่อยๆ แต่ละอันจะประกอบเข้าด้วยกัน ทับเปิดของ $\langle v, i, y, w, h \rangle$ ที่ v (v_b) เป็นค่าของพิกเซล (x, y) เป็นพิกัดของ กรอบสี่เหลี่ยมย่อยที่สัมพันธ์กับมุมด้านบนซ้ายของกรอบและ (w, h) เป็นความกว้างความสูงของกรอบ สี่เหลี่ยมย่อย โคลเอนต์สามารถแสดงกรอบที่มาจากต้นฉบับได้โดยการวาดเดิมกรอบของค่าพิกเซลพื้น หลังให้เต็มและเมื่อการวาดเดิมกรอบนั้นถูกเติมเต็มและตรงกับกรอบสี่เหลี่ยมย่อย

5.4.4 การเข้ารหัสแบบโคอาร์อาร์อี (CoRRE encoding)

การเข้ารหัสแบบโคอาร์อาร์อีเป็นส่วนที่เพิ่มขึ้นมาจากการเข้ารหัสแบบอาร์อาร์อี ที่สามารถ รับประกันได้ว่ากรอบที่ส่งออกไปจะมีขนาดใหญ่ที่สุดไม่เกิน 255×255 พิกเซล ถ้าเซิร์ฟเวอร์ต้องการส่ง กรอบที่มีขนาดใหญ่กว่านี้ก็จะทำการแยกมันออกเป็นชิ้นๆ และส่งออกมาเป็นกรอบอาร์เอฟบีเล็กๆ จำนวน หลายๆ ครั้ง ภายในแต่ละส่วนของกรอบสี่เหลี่ยมเล็กๆ เหล่านั้นจะมีหนึ่งไบต์ที่สามารถจะเอาไปใช้ในการ แทนมิติของกรอบสี่เหลี่ยมย่อยๆ เหล่านี้ สำหรับในคอมพิวเตอร์ทั่วไปผลลัพธ์จะดีกว่าการบีบอัดข้อมูล ด้วยวิธีอาร์อาร์อี ในความเป็นจริงแล้วการบีบอัดข้อมูลที่ดีที่สุดเมื่อเรากำหนดขนาดของกล่องสี่เหลี่ยมให้ มีขนาดสูงสุด 48×48 นั้นเป็นเพราะว่ากรอบข้อมูลไม่ได้ทำการเข้ารหัสไว้ (โดยปกติแล้วข้อมูลภายในจะ เป็นข้อมูลประเภทรูปภาพ) จะถูกส่งออกเป็นแบบหยาบ ถ้าอันไหนทำการเข้ารหัสไว้ก็จะถูกส่งออกไป เป็นแบบโคอาร์อาร์อี ตัวที่เล็กที่สุดของขนาดกรอบที่ใหญ่ที่สุดเป็นตัวกำหนดการตัดสินใจ ถ้าเป็น อาร์ อาร์อี กรอบเดิมทั้งหมดจะต้องถูกส่งทั้งในแบบ อาร์อาร์อี และแบบหยาบ อย่างไรก็ตามเนื่องจากว่าในการ ส่งแต่ละครั้งก็จะมีค่าไส้หุ้ย ดังนั้นการส่งแบบให้กรอบมีขนาดใหญ่ก็จะจึงมีขนาดของข้อมูลโดยรวมได้

(ดังนั้นจึงสามารถเพิ่มจำนวนของกรอบอาร์เอฟบีเพิ่มขึ้นได้) ทั้งหมดนี้เป็นการคิดในกรณีที่เลวร้ายที่สุดก่อน

5.4.5 การเข้ารหัสแบบเฮกซ์ไทล์ (hextile encoding)

เฮกซ์ไทล์เป็นการปรับปรุงแนวความคิดของ โคอาร์อาร์อี ใหม่ จากกรอบข้อมูลเดิมจะถูกแบ่งออกเป็น 16X16 ชิ้น และใช้ข้อมูล 4 บิตสำหรับแทนแต่ละกรอบสี่เหลี่ยมย่อย และ 16 บิตในการแทนข้อมูลของกรอบรวมจะไม่เหมือนกับแบบ โคอาร์อาร์อี แต่ละชิ้นจะไม่อยู่บนระดับบนของกรอบอาร์เอฟบี เมื่อทำการแตกกรอบตัวเดิมออกเป็นแผ่นย่อยๆวิธีนี้เรียกว่า การทำพรีดีเทอร์มาย นั้นหมายความว่า ตำแหน่งและขนาดของแต่ละชิ้นจะไม่มีรูปแบบกำหนดที่แน่นอนตายตัว องค์ประกอบของการเข้ารหัสของแผ่นข้อมูลย่อยที่แตกออกมาเป็นชิ้นนี้จะทำให้ง่ายในลำดับของการทำพรีดีเทอร์มาย ลำดับของชิ้นส่วนย่อยจะเริ่มต้นจากด้านซ้ายมุมบนแล้วไปทางขวาจากบนลงล่างตามลำดับ ถ้าความกว้างของกรอบหารด้วย 16 ไม่ลงตัวให้เอาความกว้างของชิ้นส่วนสุดท้ายในแต่ละแถวแทนจะทำให้ได้เล็กที่สุด ในทางของคอลัมน์ก็ทำเช่นเดียวกันหากความสูงของมันหารด้วย 16 ไม่ลงตัวก็ให้เอาแถวสุดท้ายมาทำเช่นเดียวกัน

แต่ละชิ้นส่วนย่อยจะถูกเข้ารหัสข้อมูลฟิสิกส์เป็นแบบหยาบหรืออาจจะเป็นแบบอาร์อาร์อี แล้วแต่ข้อกำหนดโดยจะถูกกำหนดโดยไบต์ที่บอกชนิดของแต่ละชิ้นส่วนย่อย แต่ละชิ้นส่วนย่อยจะเก็บค่าฟิสิกส์ของพื้นหลังไว้อย่างไรก็ตามค่าของข้อมูลฟิสิกส์จะไม่มีค่าจำเป็นมากนักหากมันมีค่าพื้นหลังเช่นเดียวกับชิ้นก่อนหน้าและค่าฟิสิกส์ของกรอบย่อยมีค่าเหมือนหมด เราสามารถกำหนดค่าหนึ่งแทนจำนวนชิ้นย่อยทั้งหมดที่ซ้ำกันแทนเพื่อลดจำนวนข้อมูลลงได้ ส่วนข้อมูลที่เป็นภาพพื้นหน้าสามารถนำมาจากชิ้นส่วนก่อนหน้าได้

5.5 เมสเสจโพรโตคอล (Protocol Messages)

อาร์เอฟบีโพรโตคอลสามารถทำงานบนระดับชั้นทรานสปอร์ตที่เชื่อถือได้ทั้งที่เป็นแบบไบต์สตรีม (byte-stream) หรือแบบเมสเสจ (Messages base) ในโพรโตคอลนี้มีขั้นตอนใหญ่ๆคือ ในขั้นตอนการเริ่มต้นการเชื่อมต่อ ที่เรียกว่า “ อินนิเชียลแฮนด์เชคกิ้ง (initial handshaking) “ ซึ่งกำหนดโดยโพรโตคอลที่ใช้ในการโต้ตอบปกติทั่วไป

อินนิเชียลแฮนด์เชคกิ้งประกอบด้วย โพรโตคอลเวอร์ชัน (ProtocolVersion), การรับรองสิทธิ (Authentication), การเริ่มต้นการเชื่อมต่อของไคลเอนต์ (ClientInitialisation), การเริ่มต้นการเชื่อมต่อของเซิร์ฟเวอร์ (ServerInitialisation) ควรจำไว้ว่าทั้งไคลเอนต์และเซิร์ฟเวอร์จะส่งเมสเสจ ProtocolVersion มาด้วย

โพรโตคอลจะทำงานในสถานะปกติหลังจากมีการส่งอินนิเชียลไลเซชัน (initialisation) เมสเสจ หลังจากขั้นตอนนี้ไคลเอนต์ก็จะสามารถส่งเมสเสจอะไรก็ได้ตามที่มันต้องการหรืออาจรับเมสเสจจากเซิร์ฟเวอร์ก็ได้ เมสเสจเหล่านี้จะต้องเริ่มต้นด้วยไบต์ที่บอกชนิดของเมสเสจ ตามด้วยข้อมูลชนิดพิเศษอื่นๆ

5.5.1 เมสเสจเริ่มต้นการเชื่อมต่อ (Initail Handshaking Messages)

5.5.1.1 โพรโตคอลเวอร์ชัน (ProtocolVersion)

การเริ่มต้นการเชื่อมต่อจะเริ่มด้วยการที่เซิร์ฟเวอร์ส่งโพรโตคอลเวอร์ชันไปยังไคลเอนต์ ในการส่งเมสเสจนี้ไคลเอนต์จะต้องรู้เลขเวอร์ชันล่าสุดของอาร์เอฟบีโพรโตคอลที่ตัวเซิร์ฟเวอร์นั้นก่อน ซึ่งมันจะสนับสนุนการทำงานนั้นอยู่แล้วโดยฝั่งไคลเอนต์จะส่งเมสเสจที่ให้เลขเวอร์ชันที่มันใช้อยู่กลับมายังเซิร์ฟเวอร์ (ซึ่งอาจจะแตกต่างกับเวอร์ชันของเซิร์ฟเวอร์ก็ได้)

อาจจะมีการตั้งใจทำให้ทั้งฝั่งไคลเอนต์และฝั่งเซิร์ฟเวอร์มีการเตรียมระดับของการเข้ากันได้ของทั้งสองฝั่งกับเวอร์ชันเก่าและควรเข้ากันได้กับเวอร์ชันใหม่ด้วย ตัวอย่างเช่น ถ้าไคลเอนต์ต้องการใช้โพรโตคอลเวอร์ชัน 3.1 เซิร์ฟเวอร์ที่ใช้เวอร์ชัน 3.0 ต้องสามารถจัดการได้โดยการไม่สนใจในสิ่งที่ไม่เข้าใจในโพรโตคอลที่ต่างกันแต่มันยังสามารถทำงานได้ในกรณีเช่นนี้ไม่เกี่ยวกับกรณีที่มีการเปลี่ยนแปลงเลขเวอร์ชันครั้งใหญ่ๆ

โพรโตคอลเวอร์ชันเมสเสจจะประกอบด้วย 12 ไบต์ ที่สามารถแปลงเป็นสตริงในแอสกีเป็นรูปแบบ "RFB xxx.yyy\n" ที่ xxx และ yyy เป็นเลขตัวหลักและตัวรองของเลขเวอร์ชัน ตามด้วยส่วนเติมเต็มด้วย 0

No. of bytes	Value
12	"RFB 003.003\n" (hex 52 46 42 20 30 30 33 2e 30 30 33 0a)

5.5.1.2 การกำหนดขอใช้สิทธิการใช้งาน (Authentication)

เมื่อจะใช้โพรโตคอลนี้เซิร์ฟเวอร์จะต้องส่งเมสเสจที่แสดงว่ามีการรับรองสิทธิที่ใช้ในการเชื่อมต่อด้วย

No. of bytes	Type	[Value]	Description
4	CARD32		<i>authentication-scheme:</i>
		0	<i>connection failed</i>
		1	<i>no authentication</i>
		2	<i>VNC authentication</i>

- Connection failed หากมีเหตุผลบางอย่างที่ทำให้การเชื่อมต่อผิดพลาด จะต้องตามด้วยสตริงที่อธิบายเหตุผลของการผิดพลาดนั้น
- No authentication ไม่มีการกำหนดการขอใช้สิทธิการใช้งาน
- VNC authentication มีการกำหนดการขอใช้สิทธิ จะตามด้วยเลข 16 ไบต์ที่ทำการสุ่มมา

No. of bytes	Type	[Value]	Description
16	CARD8		<i>challenge</i>

ไคลเอนต์จะต้องเข้ารหัสการร้องขอด้วยการเข้ารหัสแบบ DES โดยการเข้ารหัสผ่านที่ผู้ใช้ป้อนเป็นคีย์และส่งผลลัพธ์เป็น 16 ไบต์

No. of bytes	Type	[Value]	Description
16	CARD8		<i>response</i>

เซิร์ฟเวอร์จะส่งเมสเซจที่แจ้งให้ไคลเอนต์ทราบว่าการขอใช้สิทธินั้นสำเร็จ ถ้าการเชื่อมต่อสำเร็จ โพรโตคอลจะต้องตามด้วยเมสเซจ Initialisation ถ้าหากไม่สำเร็จก็จะตัดการเชื่อมต่อ

No. of bytes	Type	[Value]	Description
4	CARD32		<i>status:</i>
		0	<i>OK</i>
		1	<i>failed</i>
		2	<i>too-many</i>

ถ้าหากในการขอใช้สิทธินั้นผิดพลาดหลายครั้งติดต่อกัน โพรโตคอลไม่ควรจะอนุญาตให้มีการเชื่อมต่อจากไคลเอนต์ตัวนั้นอีก

5.5.1.3 ClientInitialisation

ในการเชื่อมต่อนั้นไคลเอนต์จะต้องส่งเมสเซจที่แสดงการเริ่มต้นการเชื่อมต่อ

No. of bytes	Type	[Value]	Description
1	CARD8		<i>shared-flag</i>

5.5.1.4 ServerInitialisation

หลังจากมีการรับเมสเซจClientInitialisationจากไคลเอนต์แล้วเซิร์ฟเวอร์จะต้องส่ง ServerInitialisation ออกไปด้วยซึ่งเมสเซจตัวนี้จะบอกความกว้าง ความยาวของเฟรมบัพเฟอร์,รูปแบบของพิกเซลและชื่อที่เกี่ยวข้องกับเดสก์ท็อปของมัน

No. of bytes	Type	[Value]	Description
2	CARD16		<i>framebuffer-width</i>
2	CARD16		<i>framebuffer-height</i>
16	PIXEL_FORMAT		<i>server-pixel-format</i>
4	CARD32		<i>name-length</i>
<i>name-length</i>	CARD8 array		<i>name-string</i>

ที่ pixel-format เป็น

No. of bytes	Type	[Value]	Description
1	CARD8		<i>bits-per-pixel</i>
1	CARD8		<i>depth</i>
1	CARD8		<i>big-endian-flag</i>
1	CARD8		<i>true-colour-flag</i>
2	CARD16		<i>red-max</i>
2	CARD16		<i>green-max</i>
2	CARD16		<i>blue-max</i>
1	CARD8		<i>red-shift</i>
1	CARD8		<i>green-shift</i>
1	CARD8		<i>blue-shift</i>
3			<i>padding</i>

server-pixel-format จะเป็นตัวกำหนดรูปแบบพิกเซลตามปกติของเซิร์ฟเวอร์ รูปแบบพิกเซลจะถูกนำมาใช้ก็ต่อเมื่อไคลเอนต์มีการเรียกร้องรูปแบบที่แปลกออกไป โดยการส่งเมสเสจ SetPixelFormat

จำนวนบิตต่อพิกเซล (bit-per-pixel) เป็นจำนวนบิตที่ใช้บนพิกเซลหนึ่งๆซึ่งมันจะต้องมากกว่าหรือเท่ากับความกว้างของกรอบที่ถูกส่งมา ซึ่งบิตเหล่านี้จะเป็นตัวบอกค่าความละเอียดของพิกเซล โดยค่านี้นี้จะต้องมีค่า 8, 16 หรือ 32 และระบบจะไม่สนับสนุนการทำงานหากมีค่าน้อยกว่า 8 บิตลงมา

ในกรณีที่บิกเอนเดียนแฟล็ก (Big-endian-flag) ไม่เป็น 0 (เป็นจริง) ถ้าพิกเซลที่ใช้หลายๆไบต์เป็นบิกเอนเดียน อาจแปลได้ว่า ใช้ 8 บิตต่อพิกเซล

ถ้า ทรู-คัลเลอร์-แฟล็ก (true-colour-flag) ไม่เป็น 0 (เป็นจริง) ให้เอาหกตัวสุดท้ายเป็นตัวกำหนดความคมชัดของสีต่างๆในแต่ละพิกเซล

เรดแมกซ์ (Red-max) เป็นค่าสูงสุดของสีแดง ($= 2^n - 1$ ที่ n เป็นจำนวนบิตของสีแดง) ข้อควรจำ ค่าเหล่านี้จะเก็บในรูปแบบบิกเอนเดียน

เรดชิฟ (Red-shift) เป็นจำนวนครั้งในการเลื่อนของสีแดง กรีนแมกซ์ (Green-max) กรีนชิฟ (Green-shift) บลูแมกซ์ (Blue-max) บลูชิฟ (Blue-shift) ก็เป็นแบบเดียวกับสีแดง ตัวอย่าง หาค่าของสีแดง (ระหว่าง 0 ถึง ค่าสูงสุดของสีแดง) จากพิกเซลที่ให้ วิธีทำดังนี้

- สลับค่าของพิกเซลให้สอดคล้องกับบิกเอนเดียนแฟล็ก (ตัวอย่างเช่น ถ้าบิกเอนเดียนเป็น 0 และลำดับไบต์ของโฮสต์เป็นบิกเอนเดียน ให้ทำการสลับค่าได้)
- ให้ทำการเลื่อนเรด-ชิฟ ไปทางขวา
- ทำการ AND ด้วย เรด-แมกซ์ (ในลำดับไบต์ของโฮสต์)

ในตอนนี้มีจำนวนน้อยหรือเกือบไม่มีเลยที่สนับสนุนการทำงานของตารางสี บางงานอาจจะทำได้แต่จะไม่สมบูรณ์ มันจะต้องเพิ่มข้อกำหนดบางอย่างดังนี้

ถ้า ทรู-คัลเลอร์-แฟล็ก เป็น 0 (เป็นเท็จ) ให้เซิร์ฟเวอร์ใช้ค่าพิกเซลที่มันจะไม่รวมเอาความเข้มของสีแดง น้ำเงิน และเขียวไว้ แต่มันจะทำการชี้ไปยังแผนที่สีแทน ซึ่งค่าที่อยู่ในแผนที่สีสามารที่จะเซตโดยไคลเอนต์โดยใช้ FixColourMapEntries เมสเสจ หรือโดยเซิร์ฟเวอร์ โดยใช้ SetColourMapEntries

แต่ FixColourMapEntries เมสเสजनั้นไม่สนับสนุนการทำงานโดยเซิร์ฟเวอร์ใดๆที่ใช้งานอยู่ในปัจจุบันนี้และแผนที่สีจะสนับสนุนการทำงานทั้งหมดโดยเซิร์ฟเวอร์แบบ X-base เท่านั้น ในความเป็นจริง

แล้วการสนับสนุนการทำงานของแผนที่สีนั้น ไคลเอนต์จำเป็นต้องมีความสามารถที่จะกำหนดค่าพิกเซลเฉพาะอื่นที่ไม่ต้องการให้เซิร์ฟเวอร์ใช้ได้ซึ่งคุณสมบัตินี้อาจถูกเพิ่มเข้าไปในเวอร์ชันถัดไปขอโปรโตคอล แต่ตอนนี้ยังไม่ได้เพิ่มคุณสมบัติข้อนี้เข้าไป

5.5.2 เมสเสจที่ส่งจากไคลเอนต์ไปยังเซิร์ฟเวอร์ (Client to server message)

5.5.2.1 SetPixelFormat

เป็นการกำหนดรูปแบบค่าของพิกเซลที่ควรจะทำการส่งไปในเมสเสจ `FramebufferUpdate` ถ้าไคลเอนต์ไม่สามารถส่งเมสเสจ `SetPixelFormat` ได้ ให้เซิร์ฟเวอร์ใช้รูปแบบค่าของพิกเซลตามที่เคยใช้อยู่ก่อนหน้านี้โดยจะถูกกำหนดในเมสเสจ `ServerInitialisation`

ณ ตอนนี้ (1998) มีคอมพิวเตอร์จำนวนน้อยที่ไม่สนับสนุนแผนที่สีหรือทำงานแต่ทำได้ไม่สมบูรณ์จึงต้องมีการเพิ่มบางส่วนดังนี้

ถ้า `true-colour-flag` เป็น 0 ให้นำเอาแผนที่สีนั้นมาใช้งานได้ ตัวไคลเอนต์เองสามารถกำหนดค่าที่จะให้คงที่ภายในแผนที่สีได้โดยใช้เมสเสจ `FixColourMapEntries` ตัวอื่นๆที่ไม่ได้ถูกทำให้มีค่าคงที่โดยไคลเอนต์สามารถที่จะถูกปรับเปลี่ยนค่าได้โดยเซิร์ฟเวอร์ได้โดยใช้เมสเสจ `SetColourMapEntries`

ในทันทีที่ไคลเอนต์มีการส่งเมสเสจนี้ออกไปแผนที่สีก็จะว่างเปล่าถ้าก่อนหน้านี้มีการกำหนดให้มันมีค่าคงที่โดยไคลเอนต์หรือเซิร์ฟเวอร์ตัวใดตัวหนึ่ง

No. of bytes	Type	[Value]	Description
1	CARD8	0	<i>message-type</i>
3			<i>padding</i>
16	PIXEL_FORMAT		<i>pixel-format</i>

ที่ `PIXEL_FORMAT` ถูกอธิบายไว้ในหัวข้อ เรื่อง Pixel Format แล้ว

No. of bytes	Type	[Value]	Description
1	CARD8		<i>bits-per-pixel</i>
1	CARD8		<i>depth</i>
1	CARD8		<i>big-endian-flag</i>
1	CARD8		<i>true-colour-flag</i>
2	CARD16		<i>red-max</i>
2	CARD16		<i>green-max</i>
2	CARD16		<i>blue-max</i>
1	CARD8		<i>red-shift</i>
1	CARD8		<i>green-shift</i>
1	CARD8		<i>blue-shift</i>
3			<i>padding</i>

5.5.2.2 FixColourMapEntries

ในเวลานี้มีคอมพิวเตอร์จำนวนน้อยที่ไม่สนับสนุนแผนที่สี หรือทำงานได้แต่ทำได้ไม่สมบูรณ์จึงต้องมีการเพิ่มบางส่วนดังนี้

เมื่อรูปแบบพิกเซลใช้แผนที่สีนี้ เมสเซจตัวนี้จะบอกเซิร์ฟเวอร์ให้กำหนดค่าพิกเซลให้เพื่อทำการแมปกับค่าในแผนที่สีสำหรับกำหนดความสว่างของRGBนั้นหมายความว่าเซิร์ฟเวอร์อาจจะไม่ได้กำหนดความสว่างของค่าRGBเองสำหรับค่าความสว่างของพิกเซลเหล่านี้เราสามารถทำการกำหนดได้โดยใช้เมสเซจ SetColourMapEntries ซึ่งถ้าไคลเอนต์มีการกำหนดแผนที่สีตายตัวมันจะต้องทำการส่งเมสเซจ FixColourMapEntries ซึ่งจะอธิบายแผนที่สีของมันทั้งหมดที่มันต้องการและเซิร์ฟเวอร์จะแปลงค่าพิกเซลทั้งหมดไปเป็นค่าต่างๆ ที่ต้องใช้ในการเข้ารหัส สำหรับแผนที่สีของไคลเอนต์ในกรณีเช่นนี้เซิร์ฟเวอร์จะไม่สามารถส่งเมสเซจ SetColourMapEntries ออกไปได้เลยเพราะไคลเอนต์ได้กำหนดไว้ก่อนแล้วเมสเซจ FixcolourMapEntries จะไม่มีการสนับสนุนการใช้งานโดยเซิร์ฟเวอร์ที่มีใช้อยู่ในตอนนี้นะ

No. of bytes	Type	[Value]	Description
1	CARD8	1	<i>message-type</i>
1			<i>padding</i>
2	CARD16		<i>first-colour</i>
2	CARD16		<i>number-of-colours</i>

number-of-colours มีลักษณะดังนี้

No. of bytes	Type	[Value]	Description
2	CARD16		<i>red</i>
2	CARD16		<i>green</i>
2	CARD16		<i>blue</i>

5.5.2.3 SetEncodings

เป็นการกำหนดชนิดของการเข้ารหัสของพิกเซลเดตาที่ส่งมาโดยเซิร์ฟเวอร์ลำดับของชนิดการเข้ารหัสจะถูกเก็บอยู่ในเมสเซจนี้โดยจะบอกโดยคร่าวๆเท่านั้นโดยไคลเอนต์จะนำค่านี้ไปใช้ในการอ้างอิงถึง(โดยการเข้ารหัสที่มาเป็นตัวแรกจะถูกกำหนดให้มีความสำคัญในการอ้างอิงถึงก่อนมากที่สุด)โดยเซิร์ฟเวอร์อาจจะใช้หรือไม่ใช้ค่าบอกใบ้ก็ได้ ข้อมูลของพิกเซลจะถูกเข้ารหัสเป็นการเข้ารหัสแบบหลายบิตถ้าหากว่าไม่มีการกำหนดไว้ก่อน

No. of bytes	Type	[Value]	Description
1	CARD8	2	<i>message-type</i>
1			<i>padding</i>
2	CARD16		<i>number-of-encodings</i>

number-of-encodings จะถูกกำหนดดังข้างล่าง

No. of bytes	Type	[Value]	Description
4	CARD32		<i>encoding-type</i>
		0	<i>raw</i> encoding
		1	<i>copy rectangle</i> encoding
		2	<i>RRE</i> encoding
		4	<i>CoRRE</i> encoding
		5	<i>hextile</i> encoding

5.5.2.4 FramebufferUpdateRequest

เป็นการเปลี่ยนแปลงค่าต่างๆภายในเครื่องเซิร์ฟเวอร์นั้นเครื่องไคลเอนต์จะสนใจเฉพาะในพื้นที่ของเฟรมบัพเฟอร์ซึ่งจะถูกกำหนดโดย ตำแหน่งของ เอ็กซ์,วาย ความกว้างและความสูง โดยปกติแล้วเซิร์ฟเวอร์จะตอบสนองกับ FramebufferUpdateRequest โดยการส่ง FramebufferUpdate อย่างไรก็ตามการส่ง FramebufferUpdate ครั้งเดียวอาจมีการส่ง FramebufferUpdateRequests เพื่อตอบสนองหลายครั้งก็ได้

เซิร์ฟเวอร์จะกำหนดให้ไคลเอนต์ทำการเก็บสำเนาของตัวที่มีการเปลี่ยนแปลงทั้งหมดในเฟรมบัพเฟอร์หมายความว่าเซิร์ฟเวอร์จำเป็นต้องส่งข้อมูลที่มีการปรับปรุงข้อมูลไปยังเซิร์ฟเวอร์บ่อยๆ

อย่างไรก็ตามถ้ามีการสูญหายของข้อมูลที่เกิดขึ้นระบบจะให้ไคลเอนต์ส่ง FramebufferUpdateRequest ใหม่และทำการเซต incremental เป็น 0 (เป็นเท็จ) จากการร้องขอข้อมูลครั้งนี้เซิร์ฟเวอร์จะส่งข้อมูลทั้งหมดของพื้นที่ที่ถูกกำหนดไว้ให้เร็วที่สุดเท่าที่จะเป็นไปได้ พื้นที่เหล่านี้อาจไม่ทำการปรับปรุงข้อมูลและส่งข้อมูลโดยใช้การเข้ารหัสแบบ copy rectangle แน่นนอน

ถ้าไคลเอนต์ไม่มีการสูญหายของข้อมูลของพื้นที่ที่มีการเปลี่ยนแปลงเกิดขึ้นให้มันทำการส่ง FramebufferUpdateRequests ใหม่และทำการเซตให้ incremental ไม่เป็น 0 (เป็นจริง) ถ้าเมื่อไหร่ก็ตามที่มีการเปลี่ยนแปลงของพื้นที่ที่ถูกกำหนดไว้ของเฟรมบัพเฟอร์ เซิร์ฟเวอร์จะส่ง FramebufferUpdate ดังนั้นอาจจะไม่มีการกำหนดระยะเวลาการทำการปรับปรุงข้อมูลระหว่าง FramebufferUpdateRequests และ FramebufferUpdate

ในกรณีของไคลเอนต์แบบทำงานเร็วมันอาจจะเลือกการส่ง FramebufferUpdateRequests ในอัตราความเร็วที่ปกติเพื่อหลีกเลี่ยงการใช้เน็ตเวิร์กเกินส่วนที่จะใช้ได้

No. of bytes	Type	[Value]	Description
1	CARD8	3	<i>message-type</i>
1	CARD8		<i>incremental</i>
2	CARD16		<i>x-position</i>
2	CARD16		<i>y-position</i>
2	CARD16		<i>width</i>
2	CARD16		<i>height</i>

5.5.2.5 KeyEvent

เป็นการกดหรือปล่อยปุ่ม โดย Down-flag จะไม่เป็น 0 (เป็นจริง) ถ้าปุ่มกำลังถูกกด และเป็น 0 (ไม่เป็นจริง) เมื่อปุ่มถูกปล่อย โดยค่าของปุ่มจะถูกกำหนดค่าโดยเรียกว่า “keysym” ค่าของปุ่มโดยปกติแล้วจะถูกกำหนดในคำรหัส ASCII โดยค่าปุ่มต่างๆ ไปมีค่าดังนี้

Key name	Keysym value	Key name	Keysym value
BackSpace	0xff08	F1	0xffbe
Tab	0xff09	F2	0xffbf
Return or Enter	0xff0d	F3	0xffc0
Escape	0xff1b	F4	0xffc1
Insert	0xff63	...	...
Delete	0xffff	F12	0xffc9
Home	0xff50	Shift (left)	0xffe1
End	0xff57	Shift (right)	0xffe2
Page Up	0xff55	Control (left)	0xffe3
Page Down	0xff56	Control (right)	0xffe4
Left	0xff51	Meta (left)	0xffe7
Up	0xff52	Meta (right)	0xffe8
Right	0xff53	Alt (left)	0xffe9
Down	0xff54	Alt (right)	0xffea

5.5.2.6 PointerEvent

เป็นตัวแสดงการเคลื่อนที่ของตัวชี้หรือการกดหรือปล่อยปุ่มของตัวชี้ โดยตัวชี้จะมีค่าตำแหน่งเอ็กซ์ ตำแหน่งวาย และสถานะปัจจุบันของปุ่มมีค่าตั้งแต่ 1 ถึง 8 โดยจะใช้บิต 0 ถึง 7 ของ button-mask ตามลำดับ โดย 0 หมายถึงขึ้น 1 หมายถึงลง (ถูกกด)

No. of bytes	Type	[Value]	Description
1	CARD8	5	<i>message-type</i>
1	CARD8		<i>button-mask</i>
2	CARD16		<i>x-position</i>
2	CARD16		<i>y-position</i>

5.5.2.7 ClientCuttext

ตัวไคลเอนต์เองจะมีตัวอักษรอยู่ในบัฟเฟอร์ตัด (cut-buffer) ของมันเอง ตัวจบบรรทัดจะแทนด้วย แอสกีค่า10 และไม่จำเป็นต้องมี carriage-return

No. of bytes	Type	[Value]	Description
1	CARD8	6	<i>message-type</i>
3			<i>padding</i>
4	CARD32		<i>length</i>
<i>length</i>	CARD8 array		<i>text</i>

5.5.3 Server to client Messages

5.5.3.1 FramebufferUpdate

เป็นการปรับปรุงเฟรมบัฟเฟอร์ ซึ่งจะประกอบจากลำดับการส่งกรอบข้อมูลพิกเซล ฟังก์ชันไคลเอนต์ จะทำการเก็บข้อมูลของพิกเซลเหล่านั้นไว้ในเฟรมบัฟเฟอร์ของมันเองโดยมันจะทำการตอบสนองกับเมส เซจFramebufferUpdateRequests จากไคลเอนต์ซึ่งอาจจะไม่มีการกำหนดช่วงเวลาการตอบสนองระหว่าง FramebufferUpdateRequests และ FramebufferUpdate

No. of bytes	Type	[Value]	Description
1	CARD8	0	<i>message-type</i>
1			<i>padding</i>
2	CARD16		<i>number-of-rectangles</i>

number-of-rectanglesจะเป็นตัวบอกจำนวนกรอบของข้อมูลพิกเซลโดยแต่ละกรอบจะประกอบด้วย

No. of bytes	Type	[Value]	Description
2	CARD16		<i>x-position</i>
2	CARD16		<i>y-position</i>
2	CARD16		<i>width</i>
2	CARD16		<i>height</i>
4	CARD32		<i>encoding-type:</i>
		0	<i>raw</i> encoding
		1	<i>copy rectangle</i> encoding
		2	<i>RRE</i> encoding
		4	<i>CoRRE</i> encoding
		5	<i>hexfile</i> encoding

ตารางข้างล่างนี้จะเป็นตัวกำหนดชนิดของการเข้ารหัส

- ในการเข้ารหัสแบบ raw ข้อมูลจะประกอบจาก ความกว้าง X ความสูง ของค่าพิกเซล ซึ่งค่าเหล่านี้จะแทนแต่ละพิกเซลในการสแกนเส้นจากซ้ายไปขวาตามลำดับ
- การเข้ารหัสแบบ copy rectangles ข้อมูลจะประกอบด้วย

No. of bytes	Type	[Value]	Description
2	CARD16		<i>src-x-position</i>
2	CARD16		<i>src-y-position</i>

- การเข้ารหัสแบบ RRE ข้อมูลจะมีส่วนหัวดังนี้

No. of bytes	Type	[Value]	Description
4	CARD32		<i>number-of-subrectangles</i>
<i>n</i>	CARD<8 <i>n</i> >		<i>background-pixel-value</i>

ที่ 8*n* คือ จำนวนบิตต่อหนึ่งพิกเซลที่มีการตกลงระหว่างไคลเอนต์และเซิร์ฟเวอร์ ทั้งที่อยู่ในเมสเซจ ServerInitialisation และ เมสเซจ SetPixelFormat โดยตัวอย่างข้างล่างจะเป็นโครงสร้างของ number-of-subrectangles

No. of bytes	Type	[Value]	Description
<i>n</i>	CARD<8 <i>n</i> >		<i>subrect-pixel-value</i>
2	CARD16		<i>x-position</i>
2	CARD16		<i>y-position</i>
2	CARD16		<i>width</i>
2	CARD16		<i>height</i>

- การเข้ารหัสแบบ CoRRE ข้อมูลจะมีส่วนหัวดังนี้

No. of bytes	Type	[Value]	Description
4	CARD32		<i>number-of-subrectangles</i>
<i>n</i>	CARD<8 <i>n</i> >		<i>background-pixel-value</i>

ที่ 8n เป็น จำนวนบิตต่อหนึ่งพิกเซลซึ่งถูกตกลงโดยไคลเอนต์และเซิร์ฟเวอร์ไว้แล้วกำหนดได้ทั้งเมสเซจ ServerInitialisation หรือ เมสเซจ SetPixelFormat โดย number-of-subrectangles มีตัวอย่างโครงสร้างดังข้างล่าง

No. of bytes	Type	[Value]	Description
n	CARD<8n>		<i>subrect-pixel-value</i>
1	CARD8		<i>x-position</i>
1	CARD8		<i>y-position</i>
1	CARD8		<i>width</i>
1	CARD8		<i>height</i>

- การเข้ารหัสแบบ hextile กรอบจะถูกแบ่งออกเป็นชิ้นๆละ 16X16 พิกเซล โดยจะเริ่มต้นจากบนซ้ายโดยจะวิ่งจากซ้ายไปขวาจากบนลงล่างตามลำดับ ถ้าความกว้างของกรอบไม่หารด้วย 16 ลงตัวให้ความกว้างของชิ้นสุดท้ายในแต่ละแถวมีขนาดเล็กลงในทำนองเดียวกันหากความสูงหารด้วย 16 ไม่ลงตัวชิ้นสุดท้ายในแต่ละแถวจะต้องมีขนาดเล็กลงเช่นเดียวกัน แต่ละชิ้นของกรอบข้อมูลย่อยจะต้องเริ่มต้นด้วยไบต์ subencoding ซึ่งภายในจะมีบิตเหล่านี้

No. of bytes	Type	[Value]	Description
1	CARD8		<i>subencoding-mask:</i>
		1	<i>Raw</i>
		2	<i>BackgroundSpecified</i>
		4	<i>ForegroundSpecified</i>
		8	<i>AnySubrects</i>
		16	<i>Subrects Coloured</i>

ถ้าบิต Raw มีการเซตบิตอื่นจะไม่ได้ได้รับความสนใจอีกเลย

BackgroundSpecified ถ้ามีการเซตบิตนี้ ค่าพิกเซลของข้อมูลชิ้นนั้นจะถูกกำหนดสีพื้นหลังไว้ด้วย

No. of bytes	Type	[Value]	Description
n	CARD<8n>		<i>background-pixel-value</i>

ที่ 8n คือจำนวนบิตต่อหนึ่งพิกเซลที่ตกลงกันไว้ระหว่างไคลเอนต์กับเซิร์ฟเวอร์โดยใช้เมสเซจ ServerInitialisation หรือเมสเซจ SetPixelFormat ชิ้นแรกของข้อมูลที่เข้ารหัสที่ไม่ใช่การเข้ารหัสแบบ raw จะต้องมีการเซตบิตนี้ก่อน ถ้าไม่มีการเซตบิตนี้ก่อนพื้นหลังจะเหมือนกับชิ้นสุดท้ายก่อนหน้านี้

ForegroundSpecified ถ้ามีการเซตบิตนี้ค่าพิกเซลที่ตามมาจะเป็นตัวกำหนดสีพื้นหน้าเพื่อใช้สำหรับทุกกรอบย่อยในชิ้นนั้น

No. of bytes	Type	[Value]	Description
n	CARD<8n>		<i>foreground-pixel-value</i>

ถ้าหากมีการเซตบิตนี้ SubrectsColoured จะต้องเป็น 0

AnySubrects ถ้าถูกเซต ค่าหนึ่งไบต์ที่ตามมาจะเป็นตัวบอกจำนวนของกรอบย่อยมีรูปแบบตามข้างล่าง

No. of bytes	Type	[Value]	Description
1	CARD8		<i>number-of-subrectangles</i>

SubrectsColoured ถ้าถูกเซตแต่ละกรอบย่อยจะถูกนำด้วยค่าของพิกเซลที่จะเป็นตัวกำหนดสีของกรอบย่อยนั้นๆ ดังนั้นกรอบย่อยจึงเป็น

No. of bytes	Type	[Value]	Description
<i>n</i>	CARD<8 <i>n</i> >		<i>subrect-pixel-value</i>
1	CARD8		<i>x-and-y-position</i>
1	CARD8		<i>width-and-height</i>

ถ้าไม่ถูกเซตกรอบย่อยทั้งหมดจะมีสีเดียวกันสำหรับสีพื้นหลังถ้าบิต ForegroundSpecified ไม่ถูกเซตให้ค่าของพื้นหน้าเหมือนกับชั้นสุดท้าย กรอบย่อยจะมีค่า

No. of bytes	Type	[Value]	Description
1	CARD8		<i>x-and-y-position</i>
1	CARD8		<i>width-and-height</i>

ตำแหน่งและขนาดของแต่ละกรอบย่อยจะถูกกำหนดในสองไบต์ทั้ง *x-and-y-position* และ *width-and-height* บิตที่มีความสำคัญสูงสุดของ *x-and-y-position* กำหนดตำแหน่งของ *X* ตัวที่มีความสำคัญน้อยที่สุดกำหนดตำแหน่ง *Y* บิตที่มีความสำคัญสูงสุดของ *width-and-height* กำหนดความกว้างบวกด้วยหนึ่ง และบิตต่ำลงมากำหนดความกว้างบวกด้วยหนึ่ง

5.5.3.2 SetColourMapEntries

ปัจจุบันนี้มีคอมพิวเตอร์จำนวนน้อยหรือเกือบไม่มีเลขที่สนับสนุนการทำงานของแผนที่สี หรืออาจจะทำงานแต่ไม่สมบูรณ์ จึงมีการเพิ่มบางอย่างลงไปดังนี้

เมื่อรูปแบบของพิกเซลเรียกใช้ แผนที่สี เมสเซนจ์จะบอกให้เครื่องไคลเอนต์กำหนดค่าของสีที่ควรจะทำแผนที่เป็นความเข้มของสี RGB โดยเซิร์ฟเวอร์อาจจะกำหนดค่าของพิกเซลซึ่งไคลเอนต์อาจจะไม่พร้อมสำหรับการเซตความสว่างของ RGB โดยใช้ FixColourMapEntries

ปัจจุบันนี้ระบบที่สนับสนุนการทำงานของแผนที่สีทั้งหมดมีเพียงเซิร์ฟเวอร์ของ X-system เท่านั้น

No. of bytes	Type	[Value]	Description
1	CARD8	1	<i>message-type</i>
1			<i>padding</i>
2	CARD16		<i>first-colour</i>
2	CARD16		<i>number-of-colours</i>

ข้างล่างจะเป็นรายละเอียดของ *number-of-colours*

No. of bytes	Type	[Value]	Description
2	CARD16		<i>red</i>
2	CARD16		<i>green</i>
2	CARD16		<i>blue</i>

5.5.3.3 Bell

คือข้อกำหนดเสียงกริ่งของเครื่องไคลเอนต์

No. of bytes	Type	[Value]	Description
1	CARD8	2	<i>message-type</i>

5.5.3.4 ServerCutText

ตัวไคลเอนต์เองจะมีตัวอักษรอยู่ในบัฟเฟอร์ตัด (cut-buffer) ของมันเอง ตัวจบบรรทัดจะแทนด้วย แอสกีค่า10 และไม่จำเป็นต้องมี carriage-return

No. of bytes	Type	[Value]	Description
1	CARD8	3	<i>message-type</i>
3			<i>padding</i>
4	CARD32		<i>length</i>
<i>length</i>	CARD8 array		<i>text</i>